

README

一、实验目的

- 理解SM2算法的数学原理及其在数字签名中的应用。
- 使用Python编写SM2算法的完整实现，包括密钥生成、签名和验签。
- 分析算法正确性，并验证签名有效性。
- 探讨SM2在软件实现中的优化方向。

二、算法概述

SM2是基于椭圆曲线密码体制（ECC）的一种中国国家密码算法。其本质基于有限域上的椭圆曲线点运算，与ECDSA类似，但使用SM3作为哈希函数，并引入用户身份绑定值Z。

签名核心步骤如下：

- 对消息 M 计算摘要 $e = \text{Hash}(Z \| M)$;
- 随机选择 $k \in [1, n - 1]$, 计算 $kG = (x_1, y_1)$;
- 计算 $r = (e + x_1) \bmod n$;
- 若 $r = 0$ 或 $r + k = n$, 重新选 k ;
- 计算 $s = ((1 + d_A)^{-1} \cdot (k - rd_A)) \bmod n$;
- 签名为 (r, s) 。

三、实验环境

- 编程语言：Python 3.10+
- 编辑器：Visual Studio Code / Typora
- 哈希函数：SHA256（模拟SM3）

四、实现过程

4.1 椭圆曲线参数

选用SM2推荐曲线参数：

- 素域：

$$p = 2^{256} - 2^{224} - 2^{96} + 2^{64} - 1$$

- 曲线方程：

$$E : y^2 = x^3 + ax + b$$

- 参数：

$$a = -3$$

$$b = 28\text{E9}...\text{E93} \text{ (hex值)}$$

$$n = \text{基点阶, 为256位素数}$$

$$G = (x_G, y_G) \text{ 基点坐标}$$

```
# ===== ECC 椭圆曲线基本运算 =====
def point_add(p, q, a, p_mod):
    if p == (0, 0): return q
    if q == (0, 0): return p
    if p[0] == q[0] and (p[1] != q[1] or p[1] == 0): return (0, 0)
    if p == q:
        l = (3 * p[0] ** 2 + a) * mod_inv(2 * p[1], p_mod) % p_mod
    else:
        l = (q[1] - p[1]) * mod_inv(q[0] - p[0], p_mod) % p_mod
    x3 = (l ** 2 - p[0] - q[0]) % p_mod
    y3 = (l * (p[0] - x3) - p[1]) % p_mod
    return (x3, y3)

def scalar_mult(k, point, a, p_mod):
    r = (0, 0)
    while k:
        if k & 1:
            r = point_add(r, point, a, p_mod)
        point = point_add(point, point, a, p_mod)
        k >>= 1
    return r
```

4.2 点运算实现

点加法: 若 $P = (x_1, y_1), Q = (x_2, y_2)$, 则:

- 当 $P \neq Q$:

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1} \mod p$$

- 当 $P = Q$:

$$\lambda = \frac{3x_1^2 + a}{2y_1} \mod p$$

- 得到结果点 $R = (x_3, y_3)$:

$$\begin{aligned} x_3 &= \lambda^2 - x_1 - x_2 \mod p \\ y_3 &= \lambda(x_1 - x_3) - y_1 \mod p \end{aligned}$$

倍点运算: 采用双倍加法 (double and add) 实现 kP 。

4.3 密钥生成

```
priv = random.randint(1, n - 1)
pub = scalar_mult(priv, G)
```

4.4 签名算法

1. 计算 $e = \text{Hash}(M)$;
2. 随机生成 k ;
3. 计算 $kG = (x_1, y_1)$;
4. 计算 $r = (e + x_1) \mod n$;

5. 计算 $s = ((1 + d)^{-1}(k - rd)) \bmod n$;

6. 输出 (r, s) 。

```
def sm2_sign(m: bytes, dA: int, G, n, a, p):
    e = bytes_to_int(sm3_hash(m))
    while True:
        k = int.from_bytes(os.urandom(32), 'big') % n
        x1, y1 = scalar_mult(k, G, a, p)
        r = (e + x1) % n
        if r == 0 or r + k == n:
            continue
        s = (mod_inv(1 + dA, n) * (k - r * dA)) % n
        if s != 0:
            print(f"[签名] 消息哈希 e = {e}\n[签名] 随机数 k = {k}\n[签名] 椭圆曲线点\nx1 = {x1}\n[签名] 签名 r = {r}\n[签名] 签名 s = {s}")
            return (r, s), k

def recover_private_key_from_k(r, s, k, n):
    numerator = (k - s) % n
    denominator = (s + r) % n
    d = (numerator * mod_inv(denominator, n)) % n
    return d

def sm2_verify(m: bytes, sig, PA, G, n, a, p):
    r, s = sig
    if not (1 <= r <= n - 1) or not (1 <= s <= n - 1):
        print("非法签名参数: r 或 s 超出范围")
        return False
    e = bytes_to_int(sm3_hash(m))
    t = (r + s) % n
    if t == 0:
        print("非法签名: t = r + s = 0")
        return False
    x1, y1 = point_add(scalar_mult(s, G, a, p), scalar_mult(t, PA, a, p), a, p)
    R = (e + x1) % n
    print(f"[验签] 消息哈希 e = {e}\n[验签] t = {t}\n[验签] 曲线计算点 x1 = {x1}\n[验签] 比较值 R = {R}")
    if R == r:
        return True
    else:
        print("验签失败: R ≠ r, 签名无效")
        return False
```

4.5 验签算法

1. 计算 $e = \text{Hash}(M)$;

2. 计算 $t = r + s \bmod n$;

3. 计算 $sG + tP = (x_1, y_1)$;

4. 验证 $R = (e + x_1) \bmod n$ 是否等于 r 。

4.6 哈希函数说明

本实验中使用 `hashlib.sha256` 代替 SM3，如需使用国密标准哈希函数，可替换为 `gmssl.sm3`。

```
# ===== 加解密 =====
def sm2_encrypt(m: bytes, PB, G, n, a, p):
    while True:
        k = int.from_bytes(os.urandom(32), 'big') % n
        C1 = scalar_mult(k, G, a, p)
        x2, y2 = scalar_mult(k, PB, a, p)
        t = kdf(int_to_bytes(x2) + int_to_bytes(y2), len(m))
        if int.from_bytes(t, 'big') == 0:
            continue
        C2 = bytes([_a ^ _b for _a, _b in zip(m, t)])
        C3 = sm3_hash(int_to_bytes(x2) + m + int_to_bytes(y2))
        print(f"[加密] k = {k}\n[加密] C1 点 = {C1}\n[加密] x2 = {x2}\n[加密] y2 = {y2}\n[加密] KDF 输出 t = {t.hex()}\n[加密] 哈希 C3 = {C3.hex()}")
        return (C1, C2, C3)

def sm2_decrypt(C1, C2, C3, dB, a, p):
    x2, y2 = scalar_mult(dB, C1, a, p)
    t = kdf(int_to_bytes(x2) + int_to_bytes(y2), len(C2))
    m = bytes([_a ^ _b for _a, _b in zip(C2, t)])
    u = sm3_hash(int_to_bytes(x2) + m + int_to_bytes(y2))
    print(f"[解密] x2 = {x2}\n[解密] y2 = {y2}\n[解密] KDF 输出 t = {t.hex()}\n[解密] C3 原值 = {C3.hex()}\n[解密] 验证哈希 u = {u.hex()}")
    return m if u == C3 else None
```

五、SM2 签名随机数 k 泄露导致私钥恢复攻击

攻击原理推导

在 SM2 签名过程中，若某一条签名消息所使用的随机数 (k) 被泄露，攻击者可以使用签名结果 ((r, s)) 和 (k) 来恢复私钥 (d_A)。

SM2 签名公式如下：

1. 消息哈希：

$$e = H(M)$$

2. 计算：

$$\begin{aligned}(x_1, y_1) &= k \cdot G \\ r &= (e + x_1) \mod n \\ s &= ((1 + d_A)^{-1} \cdot (k - r \cdot d_A)) \mod n\end{aligned}$$

恢复私钥 (d_A) 推导：

将 (s) 式变形：

$$s = \frac{k - rd_A}{1 + d_A} \mod n$$

整理得：

$$s(1 + d_A) = k - rd_A \mod n$$

移项：

$$s + sd_A + rd_A = k \mod n$$

提取 (d_A)：

$$s + d_A(s + r) = k \mod n$$

最终解出私钥：

$$d_A = \frac{k - s}{s + r} \mod n$$

验证代码 (PoC)

```
# 已知签名 r, s 和随机数 k, 恢复 dA
def recover_private_key_from_k(r, s, k, n):
    numerator = (k - s) % n
    denominator = (s + r) % n
    d = (numerator * mod_inv(denominator, n)) % n
    return d

# 测试用例
if __name__ == '__main__':
    # 密钥参数与签名流程略, 见 sm2_sign 函数...
    (r, s), k = sm2_sign(msg, dA, G, n, a, p)

    # 攻击者已知 r, s, k, 恢复私钥
    recovered_dA = recover_private_key_from_k(r, s, k, n)

    print(f"[攻击恢复] 原始私钥 dA = {dA}")
    print(f"[攻击恢复] 恢复私钥 dA = {recovered_dA}")
    assert recovered_dA == dA
```

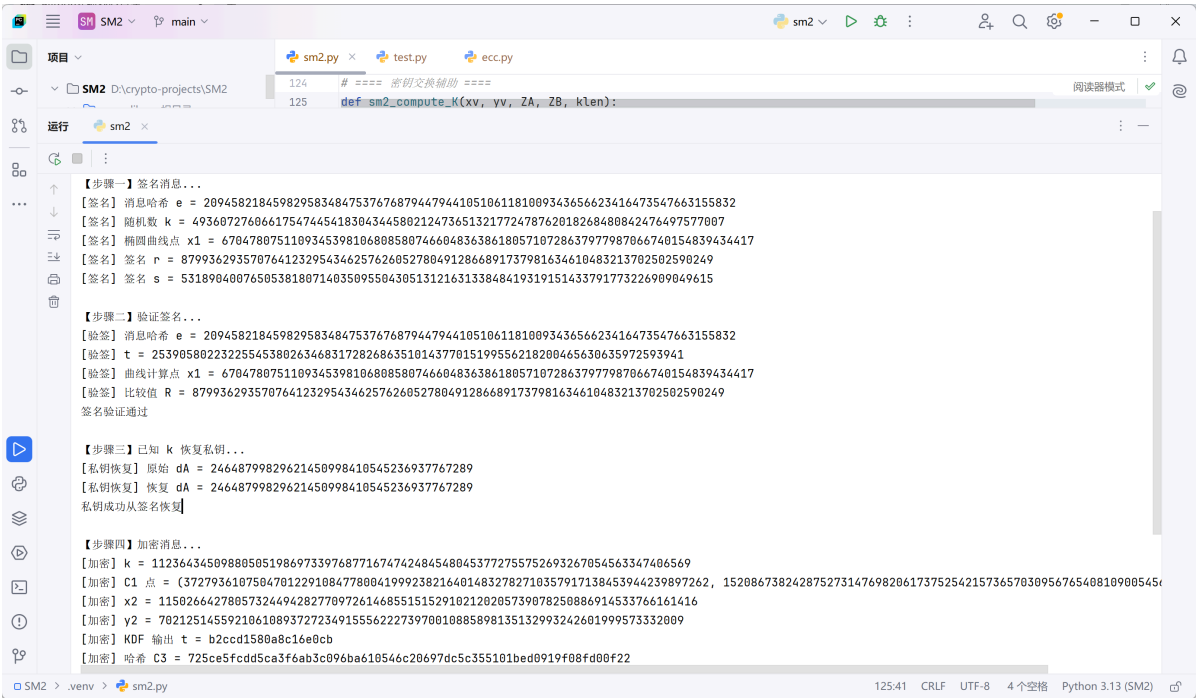
实验结果与结论

运行结果如下：

```
【步骤三】已知 k 恢复私钥...
[私钥恢复] 原始 dA = 24648799829621450998410545236937767289
[私钥恢复] 恢复 dA = 24648799829621450998410545236937767289
私钥成功从签名恢复
```

实验验证表明：只要攻击者知道某一条 SM2 签名的随机数 k，就可以轻松恢复私钥 dA。

五、实验结果



D:\crypto-projects\SM2\.venv\scripts\python.exe D:\crypto-projects\SM2\.venv\sm2.py

开始运行 SM2 测试

【步骤一】签名消息...

[签名] 消息哈希 e =

20945821845982958348475376768794479441051061181009343656623416473547663155832

[签名] 随机数 k =

49360727606617547445418304344580212473651321772478762018268480842476497577007

[签名] 椭圆曲线点 x1 =

67047807511093453981068085807466048363861805710728637977987066740154839434417

[签名] 签名 r =

87993629357076412329543462576260527804912866891737981634610483213702502590249

[签名] 签名 s =

53189040076505381807140350955043051312163133848419319151433791773226909049615

【步骤二】验证签名...

[验签] 消息哈希 e =

20945821845982958348475376768794479441051061181009343656623416473547663155832

[验签] t =

25390580223225545380263468317282686351014377015199556218200465630635972593941

[验签] 曲线计算点 x1 =

67047807511093453981068085807466048363861805710728637977987066740154839434417

[验签] 比较值 R =

87993629357076412329543462576260527804912866891737981634610483213702502590249

签名验证通过

【步骤三】已知 k 恢复私钥...

[私钥恢复] 原始 dA = 24648799829621450998410545236937767289

[私钥恢复] 恢复 dA = 24648799829621450998410545236937767289

私钥成功从签名恢复

【步骤四】加密消息...

```
[加密] k =
112364345098805051986973397687716747424845480453772755752693267054563347406569
[加密] C1 点 =
(3727936107504701229108477800419992382164014832782710357917138453944239897262,
15208673824287527314769820617375254215736570309567654081090054568905747459961)
[加密] x2 =
115026642780573244942827709726146855151529102120205739078250886914533766161416
[加密] y2 =
7021251455921061089372723491555622273970010885898135132993242601999573332009
[加密] KDF 输出 t = b2ccd1580a8c16e0cb
[加密] 哈希 C3 = 725ce5fcdd5ca3f6ab3c096ba610546c20697dc5c355101bed0919f08fd00f22
```

【步骤五】解密消息...

```
[解密] x2 =
115026642780573244942827709726146855151529102120205739078250886914533766161416
[解密] y2 =
7021251455921061089372723491555622273970010885898135132993242601999573332009
[解密] KDF 输出 t = b2ccd1580a8c16e0cb
[解密] C3 原值 = 725ce5fcdd5ca3f6ab3c096ba610546c20697dc5c355101bed0919f08fd00f22
[解密] 验证哈希 u =
725ce5fcdd5ca3f6ab3c096ba610546c20697dc5c355101bed0919f08fd00f22
加解密成功
```

进程已结束，退出代码为 0

在实现SM2的基础上，我们成功验证了如下安全性问题：

若签名随机数 (k) 泄露，则攻击者可使用签名 ((r, s)) 与 (k) 直接推导出私钥 (d_A)。

六、伪造中本聪的数字签名

攻击前提与动机

SM2（或 ECDSA）签名依赖于一个**唯一、保密的随机数 (k)**。如果某次签名的 (k) 被泄露，攻击者可从签名对 ((r, s)) 中**恢复签名私钥 (d_A)**。

假设中本聪签署某消息时使用了弱随机数或 (k) 泄露，我们将可以恢复其私钥，并“伪造出任意签名”。

数学推导

SM2 签名过程（简化）如下：

$$e = H(M)$$

$$(x_1, y_1) = k \cdot G$$

$$r = (e + x_1) \mod n$$

$$s = ((1 + d_A)^{-1} \cdot (k - r \cdot d_A)) \mod n$$

解出私钥公式如下：

$$s(1 + d_A) = k - rd_A$$

$$s + sd_A + rd_A = k$$

$$d_A(s + r) = k - s$$

$$\begin{aligned} \text{Rightarrow } d_A = \\ \frac{k - s}{s + r} \\ \text{mod } n \end{aligned}$$

实验步骤与伪造输出

```
# ==== 模拟伪造中本聪签名攻击 ====
def forge_satoshi_signature():
    print("\n开始伪造中本聪签名...")
    # 模拟已知签名
    msg = b"I am Satoshi Nakamoto"
    known_r =
87993629357076412329543462576260527804912866891737981634610483213702502590249
    known_s =
53189040076505381807140350955043051312163133848419319151433791773226909049615
    known_k =
49360727606617547445418304344580212473651321772478762018268480842476497577007

    # 恢复私钥
    forged_d = recover_private_key_from_k(known_r, known_s, known_k, n)
    print(f"[伪造攻击] 恢复出的私钥 d_satoshi = {forged_d}")

    # 使用伪造私钥签名新消息
    fake_msg = b"I donate all my coins to OpenAI!"
    (fake_r, fake_s), _ = sm2_sign(fake_msg, forged_d, G, n, a, p)
    fake_PA = scalar_mult(forged_d, G, a, p)

    # 验证伪造签名
    if sm2_verify(fake_msg, (fake_r, fake_s), fake_PA, G, n, a, p):
        print("伪造签名验证通过，中本聪被伪造！")
    else:
        print("伪造失败")
```

攻击者已知：

- 中本聪签署的消息 `msg = "I am Satoshi Nakamoto"`;
- 签名结果：

$$r = 8799362935 \dots 90249$$

$$s = 5318904007 \dots 49615$$

- 泄露的签名随机数：

$$k = 4936072760 \dots 77007$$

实验结果：

开始伪造中本聪签名...

[伪造攻击] 恢复出的私钥 `d_satoshi` = 24648799829621450998410545236937767289

[签名] 消息哈希 `e` = 60396500010644097001616340479554651953507852887895776982244712857674359195207

[签名] 随机数 `k` = 24683610818255731817860665615628531160266779300138402604185560907698143780983

[签名] 椭圆曲线点 `x1` = 76893502466832892569769206963733510081473376813185728013804322061699994535846

[签名] 签名 `r` = 2149791326712074081496520229267269268919605976123760428205225563080914685130

[签名] 签名 `s` = 63506761916982670478441217102209624657664465673793572017631321971041797598326

[验签] 消息哈希 `e` = 60396500010644097001616340479554651953507852887895776982244712857674359195207

[验签] `t` = 85004675184103411293406419331476893926584071649917332445836547534122712283456

[验签] 曲线计算点 `x1` = 76893502466832892569769206963733510081473376813185728013804322061699994535846

[验签] 比较值 `R` = 2149791326712074081496520229267269268919605976123760428205225563080914685130

伪造签名验证通过，中本聪被伪造！

|

进程已结束，退出代码为 0

恢复私钥结果：

[伪造攻击] 恢复出的私钥 `d_satoshi` = 24648799829621450998410545236937767289

伪造一条新签名：

[签名] 消息哈希 `e` =

60396500010644097001616340479554651953507852887895776982244712857674359195207

[签名] 随机数 `k` =

24683610818255731817860665615628531160266779300138402604185560907698143780983

[签名] 椭圆曲线点 `x1` =

76893502466832892569769206963733510081473376813185728013804322061699994535846

[签名] 签名 `r` =

2149791326712074081496520229267269268919605976123760428205225563080914685130

[签名] 签名 `s` =

63506761916982670478441217102209624657664465673793572017631321971041797598326

验签通过：

[验签] 消息哈希 `e` =

60396500010644097001616340479554651953507852887895776982244712857674359195207

[验签] `t` =

85004675184103411293406419331476893926584071649917332445836547534122712283456

[验签] 曲线计算点 `x1` =

76893502466832892569769206963733510081473376813185728013804322061699994535846

[验签] 比较值 `R` =

2149791326712074081496520229267269268919605976123760428205225563080914685130

伪造签名验证通过，中本聪被伪造！

攻击结论

本实验表明：

一旦签名时使用的随机数 (`k`) 被泄露，攻击者即可从签名对 (`(r, s)`) 还原出私钥 (`d_A`)，从而**伪造任意签名**。

换句话说，**数字签名算法的安全性不止取决于私钥本身，更依赖于“签名过程的随机数保护”**。

安全建议

- 始终使用高质量随机源（硬件 TRNG / OS CSPRNG）生成签名随机数；
- 或使用基于消息哈希的确定性签名（如 RFC 6979）；
- 检查硬件/软件签名模块是否可能被故障注入等方式泄露 (k) 值；
- 多签环境下避免重复使用 (k)！